

LAPORAN PRAKTIKUM STRUKTUR DATA IMPLEMENTASI SORTING dan GUI PADA JAVA

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026**

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum "Implementasi Sorting dan GUI pada Java" ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan dan bimbingan selama proses praktikum berlangsung.

Laporan ini disusun sebagai dokumentasi kegiatan praktikum yang bertujuan untuk memahami algoritma sorting seperti Bubble Sort, Selection Sort, dan Insertion Sort, serta implementasinya dalam antarmuka grafis (GUI) menggunakan Java Swing, mulai dari perancangan komponen dasar hingga integrasi algoritma ke dalam aplikasi secara visual.

Saya menyadari laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai implementasi sorting dan GUI pada Java.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	
DAFTAR ISI.....	
BAB I PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan	
1.3 Manfaat.....	
BAB II PEMBAHASAN	
2.1 InsertionSort_2511533001.....	
2.2 SelectionSort_2511533001	
2.3 BubleSort_2511533001	
2.4 InsertionSortGUI_2511533001	
2.5 Tugas	
BAB III KESIMPULAN	
3.1 Kesimpulan	
3.2 Saran.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Setiap program yang bekerja dengan data hampir pasti membutuhkan proses pengurutan di dalamnya. Sorting bukan sekadar fitur tambahan, melainkan bagian penting dari logika pemrograman yang memengaruhi seberapa cepat dan efisien sebuah sistem bekerja. Bayangkan sebuah aplikasi yang menampilkan ratusan data tanpa urutan, tentu akan menyulitkan pengguna dalam membaca maupun mencari informasi yang dibutuhkan.

Java sendiri menyediakan berbagai cara untuk mengimplementasikan algoritma sorting, mulai dari yang paling sederhana seperti Bubble Sort hingga yang lebih terstruktur seperti Selection Sort dan Insertion Sort. Masing-masing memiliki cara kerja yang berbeda dan layak untuk dipahami, bukan hanya dihafal.

Namun pemahaman algoritma saja tidak cukup jika program hanya bisa dijalankan lewat konsol. Di sinilah peran GUI menjadi penting. Dengan Java Swing, hasil kerja algoritma sorting bisa ditampilkan secara visual dalam sebuah antarmuka yang lebih nyaman digunakan. Praktikum ini menjadi kesempatan untuk menggabungkan keduanya, memahami algoritma sekaligus belajar membangun tampilan yang fungsional.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Bubble Sort, Selection Sort, dan Insertion Sort.
2. Mengetahui perbedaan logika pengurutan dari masing-masing algoritma.
3. Mempelajari komponen dasar Java Swing dalam membangun antarmuka grafis.
4. Mengintegrasikan algoritma sorting ke dalam aplikasi GUI berbasis Java.
5. Melatih kemampuan berpikir logis dalam merancang program yang terstruktur dan interaktif.

1.3 Manfaat

1. Memperkuat pemahaman tentang algoritma sorting dan penerapannya dalam Java.
2. Melatih kemampuan membangun antarmuka pengguna yang sederhana menggunakan Java Swing.
3. Memberikan pengalaman langsung dalam menggabungkan logika algoritma dengan tampilan visual.
4. Menambah wawasan tentang pentingnya sorting dalam pengembangan aplikasi yang bekerja dengan data.

BAB II

PEMBAHASAN

2.1 InsertionSort_2511533001

```
1 package pekan7_2511533001;
2
3 public class InsertionSort_2511533001 {
4     public static void insertionSort_2511533001(int[] arr_3001) {
5         int n_3001 = arr_3001.length;
6         for(int i_3001 = 1; i_3001 < n_3001; i_3001++) {
7             int key_3001 = arr_3001[i_3001];
8             int j_3001 = i_3001 - 1;
9             while(j_3001 >= 0 && arr_3001[j_3001] > key_3001) {
10                 arr_3001[j_3001 + 1] = arr_3001[j_3001];
11                 j_3001--;
12             }
13             arr_3001[j_3001 + 1] = key_3001;
14         }
15     }
16     public static void main(String[] args) {
17         int arr_3001[] = {23, 78, 45, 8, 32, 56, 1};
18         int n_3001 = arr_3001.length;
19         System.out.printf("array yang belum terurut:\n");
20         for(int i_3001 = 0; i_3001 < n_3001; i_3001++)
21             System.out.print(arr_3001[i_3001] + " ");
22         System.out.println("");
23         insertionSort_2511533001(arr_3001);
24         System.out.printf("array yang terurut:\n");
25         for(int i_3001=0; i_3001 < n_3001; i_3001++)
26             System.out.print(arr_3001[i_3001] + " ");
27         System.out.println("");
28     }
29 }
```

Penjelasan:

Class ini mengimplementasikan algoritma Insertion Sort, cara kerjanya mirip seperti orang menyusun kartu di tangan. Setiap elemen diambil satu per satu lalu disisipkan ke posisi yang benar di antara elemen-elemen sebelumnya yang sudah terurut.

Di dalam method insertionSort_2511533001, variabel key_3001 menyimpan elemen yang sedang diproses. Kemudian elemen-elemen sebelumnya yang nilainya lebih besar dari key_3001 digeser satu posisi ke kanan, memberi ruang untuk key_3001 disisipkan di tempat yang sesuai. Proses ini diulang dari indeks 1 sampai akhir array.

Output:

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2 SelectionSort_2511533001

```
1 package pekan7_2511533001;
2
3 public class SelectionSort_2511533001 {
4     public static void selectionSort_2511533001(int[] arr_3001) {
5         int n_3001 = arr_3001.length;
6         for(int i_3001 = 0; i_3001 < n_3001; i_3001++) {
7             int minIndex_3001 = i_3001;
8             for(int j_3001 = i_3001 + 1; j_3001 < n_3001; j_3001++) {
9                 if(arr_3001[j_3001] < arr_3001[minIndex_3001]) {
10                     minIndex_3001 = j_3001;
11                 }
12             }
13             int temp_3001 = arr_3001[i_3001];
14             arr_3001[i_3001] = arr_3001[minIndex_3001];
15             arr_3001[minIndex_3001] = temp_3001;
16         }
17     }
18     public static void main(String[] args) {
19         int arr_3001[] = {23, 78, 45, 8, 32, 56, 1};
20         int n_3001 = arr_3001.length;
21         System.out.printf("array yang belum terurut:\n");
22         for(int i_3001 = 0; i_3001 < n_3001; i_3001++)
23             System.out.print(arr_3001[i_3001] + " ");
24         System.out.println("");
25         selectionSort_2511533001(arr_3001);
26         System.out.printf("array yang terurut:\n");
27         for(int i_3001=0; i_3001 < n_3001; i_3001++)
28             System.out.print(arr_3001[i_3001] + " ");
29         System.out.println("");
30     }
31 }
```

Penjelasan:

Class ini mengimplementasikan algoritma Selection Sort. Berbeda dari Bubble Sort yang menukar elemen setiap kali ada yang lebih kecil, Selection Sort lebih sabar, ia mencari dulu elemen terkecil dari sisa array yang belum terurut, baru kemudian menukarnya ke posisi yang tepat.

Di dalam method selectionSort_2511533001, perulangan luar menentukan posisi yang sedang diisi, sementara perulangan dalam mencari indeks elemen terkecil mulai dari posisi berikutnya. Setelah indeks minimum ditemukan lewat variabel minIndex_3001, elemen di posisi tersebut ditukar dengan elemen di posisi perulangan luar. Prosesnya lebih sedikit melakukan penukaran dibanding Bubble Sort, meski jumlah pembandingannya sama saja.

Output:

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.3 BubleSort_2511533001

```
1 package pekan7_2511533001;
2
3 public class BubleSort_2511533001 {
4     public static void bubbleSort_2511533001(int[] arr_3001) {
5         int n_3001 = arr_3001.length;
6         for(int i_3001 = 0; i_3001 < n_3001; i_3001++) {
7             for(int j_3001 = 0; j_3001 < n_3001 - i_3001 - 1; j_3001++) {
8                 if(arr_3001[j_3001] > arr_3001[j_3001 + 1]) {
9                     int temp_3001 = arr_3001[j_3001];
10                    arr_3001[j_3001] = arr_3001[j_3001 + 1];
11                    arr_3001[j_3001 + 1] = temp_3001;
12                }
13            }
14        }
15    }
16    public static void main(String[] args) {
17        int arr_3001[] = {23, 78, 45, 8, 32, 56, 1};
18        int n_3001 = arr_3001.length;
19        System.out.printf("array yang belum terurut:\n");
20        for(int i_3001 = 0; i_3001 < n_3001; i_3001++)
21            System.out.print(arr_3001[i_3001] + " ");
22        System.out.println("");
23        bubbleSort_2511533001(arr_3001);
24        System.out.printf("array yang terurut menggunakan BubleSort:");
25        for(int i_3001 = 0; i_3001 < n_3001; i_3001++)
26            System.out.print(arr_3001[i_3001] + " ");
27        System.out.println("");
28    }
29 }
```

Penjelasan:

Class ini mengimplementasikan algoritma Bubble Sort, salah satu algoritma pengurutan paling dasar. Cara kerjanya sederhana, elemen-elemen array dibandingkan berpasangan secara berurutan, jika elemen kiri lebih besar dari elemen kanan maka keduanya ditukar. Proses ini terus berulang sampai seluruh array terurut dari nilai terkecil ke terbesar.

Di dalam method bubbleSort_2511533001, terdapat dua perulangan bersarang. Perulangan luar mengontrol berapa kali proses pembandingan dilakukan, sedangkan perulangan dalam yang benar-benar membandingkan dan menukar elemen. Setiap kali perulangan luar selesai satu putaran, elemen terbesar sudah pasti berada di posisi paling akhir, makanya batas perulangan dalam terus dikurangi dengan $n - i - 1$. Di method main, array {23, 78, 45, 8, 32, 56, 1} dipakai sebagai data uji, ditampilkan sebelum dan sesudah proses sorting berjalan.

Output:

array yang belum terurut:

23 78 45 8 32 56 1

array yang terurut menggunakan BubbleSort: 1 8 23 32 45 56 78

2.4 InsertionSortGUI_2511533001

```
1 package pekan7_2511533001;
2
3 import java.awt.BorderLayout;
4
5 public class InsertionSortGUI_2511533001 extends JFrame {
6
7     private static final long serialVersionUID = 1L;
8     private int[] array_3001;
9     private JLabel[] labelArray_3001;
10    JButton stepButton_3001;
11    JButton resetButton_3001;
12    JButton setButton_3001;
13    private JTextField inputField_3001;
14    private JPanel panelArray_3001;
15    private JTextArea stepArea_3001;
16
17    private int i_3001 = 1, j_3001;
18    private boolean sorting_3001 = false;
19    private int stepCount_3001 = 1;
20
21    public InsertionSortGUI_2511533001() {
22        setTitle("Insertion Sort Langkah per Langkah");
23        setSize(750, 400);
24        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
25        setLocationRelativeTo(null);
26        getContentPane().setLayout(new BorderLayout());
27
28        // FIX 3: inputField dibuat di sini dan ditambahkan ke inputPanel (bukan ke scrollPane)
29        JPanel inputPanel_3001 = new JPanel(new FlowLayout());
30        inputField_3001 = new JTextField(30);
31        setButton_3001 = new JButton("Set Array");
32
33        inputPanel_3001.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
34        inputPanel_3001.add(inputField_3001);
35        inputPanel_3001.add(setButton_3001);
36
37        panelArray_3001 = new JPanel();
38        panelArray_3001.setLayout(new FlowLayout());
39
40        JPanel controlPanel_3001 = new JPanel();
41        stepButton_3001 = new JButton("Langkah Selanjutnya");
42        resetButton_3001 = new JButton("Reset");
43        stepButton_3001.setEnabled(false);
44        controlPanel_3001.add(stepButton_3001);
45        controlPanel_3001.add(resetButton_3001);
46
47        stepArea_3001 = new JTextArea(8, 60);
48        stepArea_3001.setEditable(false);
49        stepArea_3001.setFont(new Font("Monospaced", Font.PLAIN, 14));
50        JScrollPane scrollPane = new JScrollPane(stepArea_3001);
51
52        getContentPane().add(inputPanel_3001, BorderLayout.NORTH);
53        getContentPane().add(panelArray_3001, BorderLayout.CENTER);
54        getContentPane().add(controlPanel_3001, BorderLayout.SOUTH);
55        getContentPane().add(scrollPane, BorderLayout.EAST);
56
57        setButton_3001.addActionListener(e -> setArrayFromInput_3001());
58        stepButton_3001.addActionListener(e -> performStep_3001());
59        resetButton_3001.addActionListener(e -> reset_3001());
60    }
61
62    private void setArrayFromInput_3001() {
63        String text_3001 = inputField_3001.getText().trim();
64        if (text_3001.isEmpty()) return;
65        String[] parts_3001 = text_3001.split(",");
66        array_3001 = new int[parts_3001.length];
67        try {
68            for (int k_3001 = 0; k_3001 < parts_3001.length; k_3001++) {
69                array_3001[k_3001] = Integer.parseInt(parts_3001[k_3001].trim());
70            }
71        } catch (NumberFormatException e) {
72            JOptionPane.showMessageDialog(this, "Masukan hanya angka dipisahkan dengan koma!", "Error",
73                JOptionPane.ERROR_MESSAGE);
74            return;
75        }
76        i_3001 = 1;
77        stepCount_3001 = 1;
78        sorting_3001 = true;
79        stepButton_3001.setEnabled(true);
80        stepArea_3001.setText("");
81        panelArray_3001.removeAll();
82        labelArray_3001 = new JLabel[array_3001.length];
83        for (int k_3001 = 0; k_3001 < array_3001.length; k_3001++) {
84            labelArray_3001[k_3001] = new JLabel(String.valueOf(array_3001[k_3001]));
85            labelArray_3001[k_3001].setFont(new Font("Arial", Font.BOLD, 24));
86            labelArray_3001[k_3001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
87            labelArray_3001[k_3001].setPreferredSize(new Dimension(50, 50));
88            labelArray_3001[k_3001].setHorizontalAlignment(SwingConstants.CENTER);
89            panelArray_3001.add(labelArray_3001[k_3001]);
90        }
91        panelArray_3001.revalidate();
92        panelArray_3001.repaint();
93    }
94
95    private void performStep_3001() {
96        if (i_3001 < array_3001.length && sorting_3001) {
97            int key_3001 = array_3001[i_3001];
98            j_3001 = i_3001 - 1;
99
100            StringBuilder stepLog_3001 = new StringBuilder();
101            stepLog_3001.append("Langkah ").append(stepCount_3001).append(": ").append(key_3001).append("\n");
102            stepArea_3001.append(stepLog_3001);
103            stepCount_3001++;
104            i_3001++;
105        }
106    }
107 }
```

```

119 while (j_3001 >= 0 && array_3001[j_3001] > key_3001) {
120     array_3001[j_3001 + 1] = array_3001[j_3001];
121     j_3001--;
122 }
123
124 // FIX 1: j_3001 + 1 bukan j_3001 = 1
125 array_3001[j_3001 + 1] = key_3001;
126
127 updateLabels_3001();
128
129 stepLog_3001.append("Hasil: ").append(arrayToString_3001(array_3001)).append("\n\n");
130
131 // FIX 2: append log ke stepArea agar muncul di layar
132 stepArea_3001.append(stepLog_3001.toString());
133
134 i_3001++;
135 stepCount_3001++;
136
137 if (i_3001 == array_3001.length) {
138     sorting_3001 = false;
139     stepButton_3001.setEnabled(false);
140     JOptionPane.showMessageDialog(this, "Sorting selesai!");
141 }
142 }
143
144 private void updateLabels_3001() {
145     for (int k = 0; k < array_3001.length; k++) {
146         labelArray_3001[k].setText(String.valueOf(array_3001[k]));
147     }
148 }
149
150 private void reset_3001() {
151     // TODO: Field 3001 - set text //
152     panelArray_3001.removeAll();
153     panelArray_3001.revalidate();
154     panelArray_3001.repaint();
155     stepArea_3001.setText("");
156     stepButton_3001.setEnabled(false);
157     sorting_3001 = false;
158     i_3001 = 1;
159     stepCount_3001 = 1;
160 }
161
162 private String arrayToString_3001(int[] arr_3001) {
163     StringBuilder sb = new StringBuilder();
164     for (int k = 0; k < arr_3001.length; k++) {
165         sb.append(arr_3001[k]);
166         if (k < arr_3001.length - 1) sb.append(", ");
167     }
168     return sb.toString();
169 }
170
171 // FIX 4: Tambah method main agar bisa dijalankan
172 public static void main(String[] args) {
173     EventQueue.invokeLater(new Runnable() {
174         public void run() {
175             try {
176                 InsertionSortGUI_2511533001 frame = new InsertionSortGUI_2511533001();
177                 frame.setVisible(true);
178             } catch (Exception e) {
179                 e.printStackTrace();
180             }
181         }
182     });
183 }
184 }
185 }

```

Penjelasan:

Class ini adalah versi visual dari Insertion Sort yang dibangun menggunakan Java Swing. Kalau tiga class sebelumnya hanya menampilkan hasil di konsol, class ini memungkinkan pengguna melihat proses pengurutan secara langsung lewat antarmuka grafis yang lebih interaktif.

Tampilan jendela dibagi menjadi beberapa bagian menggunakan BorderLayout. Di bagian atas terdapat input field tempat pengguna memasukkan angka yang dipisahkan koma, beserta tombol Set Array untuk memproses input tersebut. Setelah tombol ditekan, input dibaca, dipecah berdasarkan koma, lalu diubah menjadi array integer. Jika ada karakter yang bukan angka, program langsung menampilkan dialog error tanpa melanjutkan proses.

Di bagian tengah, setiap elemen array ditampilkan sebagai kotak label berukuran seragam dengan border hitam. Kotak-kotak ini akan terus diperbarui setiap kali satu langkah sorting dijalankan, sehingga perubahan posisi elemen bisa terlihat secara visual. Di bagian kanan terdapat area teks berjenis JTextArea yang dibungkus JScrollPane, berisi catatan tiap langkah

sorting mulai dari elemen mana yang sedang diproses hingga kondisi array setelah langkah tersebut selesai.

Tombol "Langkah Selanjutnya" di bagian bawah menjalankan tepat satu iterasi Insertion Sort setiap kali ditekan. Ini memungkinkan pengguna mengikuti alur pengurutan satu per satu tanpa semuanya langsung selesai sekaligus. Ketika seluruh elemen sudah berada di posisi yang benar, tombol otomatis dinonaktifkan dan muncul notifikasi bahwa proses sorting telah selesai. Tersedia juga tombol Reset yang mengembalikan semua komponen ke kondisi awal, sehingga pengguna bisa mencoba dengan data yang berbeda tanpa perlu menutup dan membuka ulang aplikasi.

Ouput:



Masukkan angka (pisahkan dengan koma): 25,21,99,56,34 Set Array

21 25 34 56

99

Langkah 1: Memasukkan 21
Hasil: 21, 25, 99, 56, 34

Langkah 2: Memasukkan 99
Hasil: 21, 25, 99, 56, 34

Langkah 3: Memasukkan 56
Hasil: 21, 25, 56, 99, 34

Langkah 4: Memasukkan 34
Hasil: 21, 25, 34, 56, 99

2.5 Tugas

1.Mahasiswa_2511533001

```
1 package pekan7_2511533001;
2
3 public class Mahasiswa_2511533001 {
4
5     private String nama_3001;
6     private String nim_3001;
7     private String prodi_3001;
8
9     public Mahasiswa_2511533001(String nama_3001, String nim_3001, String prodi_3001) {
10
11         this.nama_3001 = nama_3001;
12         this.nim_3001 = nim_3001;
13         this.prodi_3001 = prodi_3001;
14     }
15
16     public String getNama_3001() {
17         return nama_3001;
18     }
19
20     public String getNim_3001() {
21         return nim_3001;
22     }
23
24     public String getProdi_3001() {
25         return prodi_3001;
26     }
27
28     @Override
29     public String toString() {
30
31         return nama_3001 + " (" + nim_3001 + " - " + prodi_3001 + ")";
32     }
33 }
```

Penjelasan:

Class Mahasiswa_2511533001 digunakan untuk menyimpan data mahasiswa berupa nama, NIM, dan program studi. Constructor digunakan untuk mengisi data awal objek mahasiswa, sedangkan method getter digunakan untuk mengambil data atribut. Method toString() digunakan agar data mahasiswa dapat langsung ditampilkan pada GUI dalam bentuk teks.

2.SortingGUI_2511533001

```

1 package pekan7_2511533001;
2
3 import javax.swing.*.*;
4 import java.awt.*.*;
5 import java.util.ArrayList;
6
7 public class SortingGUI_2511533001 extends JFrame {
8
9     private ArrayList<Mahasiswa_2511533001> listMahasiswa_3001 = new ArrayList<>();
10
11     private JTextField txtNama_3001, txtNim_3001, txtProdi_3001;
12     private JTextArea txtHasil_3001;
13     private JComboBox<String> comboSort_3001;
14
15     public SortingGUI_2511533001() {
16
17         // data awal
18         listMahasiswa_3001.add(new Mahasiswa_2511533001(
19             "Zaky", "2511533001", "Informatika"));
20
21         listMahasiswa_3001.add(new Mahasiswa_2511533001(
22             "Digo", "2511523017", "Sistem Informasi"));
23
24         listMahasiswa_3001.add(new Mahasiswa_2511533001(
25             "Karim", "2511512029", "Teknik Komputer"));
26
27         listMahasiswa_3001.add(new Mahasiswa_2511533001(
28             "Endy", "2511531017", "Informatika"));
29
30         listMahasiswa_3001.add(new Mahasiswa_2511533001(
31             "Fahri", "2511511007", "Teknik Komputer"));
32
33         setTitle("Sorting GUI Mahasiswa");
34         setSize(600, 500);
35         setDefaultCloseOperation(EXIT_ON_CLOSE);
36         setLocationRelativeTo(null);
37         setLayout(new BorderLayout());
38
39         // panel input
40         JPanel panelInput_3001 = new JPanel(new GridLayout(4,2,5,5));
41
42         txtNama_3001 = new JTextField();
43         txtNim_3001 = new JTextField();
44         txtProdi_3001 = new JTextField();
45
46         panelInput_3001.add(new JLabel("Nama"));
47         panelInput_3001.add(txtNama_3001);
48
49         panelInput_3001.add(new JLabel("NIM"));
50         panelInput_3001.add(txtNim_3001);
51
52         panelInput_3001.add(new JLabel("Prodi"));
53         panelInput_3001.add(txtProdi_3001);
54
55         JButton btnTambah_3001 = new JButton("Tambah");
56         JButton btnReset_3001 = new JButton("Reset");
57
58         panelInput_3001.add(btnTambah_3001);
59         panelInput_3001.add(btnReset_3001);
60
61         add(panelInput_3001, BorderLayout.NORTH);
62
63         // text area
64         txtHasil_3001 = new JTextArea();
65         txtHasil_3001.setEditable(false);
66
67         add(new JScrollPane(txtHasil_3001), BorderLayout.CENTER);
68
69         // panel bawah
70         JPanel panelBawah_3001 = new JPanel();
71
72         String[] metode_3001 = {
73             "Insertion Sort",
74             "Selection Sort",
75             "Bubble Sort"
76         };
77
78         comboSort_3001 = new JComboBox<>(metode_3001);
79
80         JButton btnSort_3001 = new JButton("Sorting");
81
82         panelBawah_3001.add(comboSort_3001);
83         panelBawah_3001.add(btnSort_3001);
84
85         add(panelBawah_3001, BorderLayout.SOUTH);
86
87         tampilData_3001();
88
89         // tombol tambah
90         btnTambah_3001.addActionListener(e -> {
91
92             String nama_3001 = txtNama_3001.getText();
93             String nim_3001 = txtNim_3001.getText();
94             String prodi_3001 = txtProdi_3001.getText();
95
96             if(nama_3001.isEmpty() ||
97                 nim_3001.isEmpty() ||
98                 prodi_3001.isEmpty()){
99
100                 JOptionPane.showMessageDialog(
101                     null,
102                     "Isi semua data!");
103             }
104         });
105     }
106 }

```

```

103     }else{
104
105         listMahasiswa_3001.add(
106             new Mahasiswawa_2511533001(
107                 nama_3001,
108                 nim_3001,
109                 prodi_3001));
110
111         tampilData_3001();
112
113         txtNama_3001.setText("");
114         txtNim_3001.setText("");
115         txtProdi_3001.setText("");
116     }
117 });
118
119 // tombol reset
120 btnReset_3001.addActionListener(e -> {
121
122     listMahasiswa_3001.clear();
123     txtHasil_3001.setText("Data telah di-reset");
124 });
125
126 // tombol sorting
127 btnSort_3001.addActionListener(e -> {
128
129     ArrayList<Mahasiswa_2511533001> data_3001 =
130         new ArrayList<>(listMahasiswa_3001);
131
132     String pilih_3001 =
133         comboSort_3001.getSelectedItem().toString();
134
135     txtHasil_3001.setText(
136         "==== " + pilih_3001 + "====\n");
137
138     txtHasil_3001.append(
139         "Data Awal : " +
140         tampilNama_3001(data_3001) + "\n\n");
141
142     if(pilih_3001.equals("Insertion Sort")){
143
144         insertionSort_3001(data_3001);
145
146     }else if(pilih_3001.equals("Selection Sort")){
147
148         selectionSort_3001(data_3001);
149
150     }else{
151
152         bubbleSort_3001(data_3001);
153     }
154
155     txtHasil_3001.append("\n=== HASIL AKHIR ===\n");
156
157     for(Mahasiswa_2511533001 mhs_3001 : data_3001){
158
159         txtHasil_3001.append(mhs_3001 + "\n");
160     }
161 });
162 }
163
164 // tampil data
165 private void tampilData_3001() {
166
167     txtHasil_3001.setText("Data Saat Ini:\n");
168
169     for(Mahasiswa_2511533001 mhs_3001 : listMahasiswa_3001){
170
171         txtHasil_3001.append(mhs_3001 + "\n");
172     }
173 }
174
175 // tampil nama
176 private String tampilNama_3001(ArrayList<Mahasiswa_2511533001> list_3001){
177
178     String hasil_3001 = "[";
179
180     for(int i_3001 = 0; i_3001 < list_3001.size(); i_3001++){
181
182         hasil_3001 += list_3001.get(i_3001).getNama_3001();
183
184         if(i_3001 != list_3001.size()-1){
185
186             hasil_3001 += ", ";
187         }
188     }
189
190     return hasil_3001 + "]";
191 }
192
193 // insertion sort
194 private void insertionSort_3001(ArrayList<Mahasiswa_2511533001> arr_3001){
195
196     for(int i_3001 = 1; i_3001 < arr_3001.size(); i_3001++){
197
198         Mahasiswawa_2511533001 key_3001 = arr_3001.get(i_3001);
199
200         int j_3001 = i_3001 - 1;
201
202         while(j_3001 >= 0 &&
203             arr_3001.get(j_3001).getNama_3001()
204                 .compareToIgnoreCase(

```

```

205         key_3001.getNama_3001()) > 0){
206
207         arr_3001.set(j_3001 + 1,
208             arr_3001.get(j_3001));
209
210         j_3001--;
211     }
212
213     arr_3001.set(j_3001 + 1, key_3001);
214
215     txtHasil_3001.append(
216         "Langkah " + i_3001 + " : " +
217         tampilNama_3001(arr_3001) + "\n");
218     }
219 }
220
221 // selection sort
222 private void selectionSort_3001(ArrayList<Mahasiswa_2511533001> arr_3001){
223
224     for(int i_3001 = 0; i_3001 < arr_3001.size()-1; i_3001++){
225
226         int min_3001 = i_3001;
227
228         for(int j_3001 = i_3001 + 1;
229             j_3001 < arr_3001.size();
230             j_3001++){
231
232             if(arr_3001.get(j_3001).getNama_3001()
233                 .compareToIgnoreCase(
234                     arr_3001.get(min_3001)
235                         .getNama_3001()) < 0){
236
237                 min_3001 = j_3001;
238             }
239
240             Mahasiswa_2511533001 temp_3001 = arr_3001.get(min_3001);
241
242             arr_3001.set(min_3001, arr_3001.get(i_3001));
243             arr_3001.set(i_3001, temp_3001);
244
245             txtHasil_3001.append(
246                 "Pass " + (i_3001+1) + " : " +
247                 tampilNama_3001(arr_3001) + "\n");
248         }
249     }
250 }
251
252 // bubble sort
253 private void bubbleSort_3001(ArrayList<Mahasiswa_2511533001> arr_3001){
254
255     for(int i_3001 = 0; i_3001 < arr_3001.size()-1; i_3001++){
256
257         for(int j_3001 = 0;
258             j_3001 < arr_3001.size()-i_3001-1;
259             j_3001++){
260
261             if(arr_3001.get(j_3001).getNama_3001()
262                 .compareToIgnoreCase(
263                     arr_3001.get(j_3001+1)
264                         .getNama_3001()) > 0){
265
266                 Mahasiswa_2511533001 temp_3001 =
267                     arr_3001.get(j_3001);
268
269                 arr_3001.set(j_3001,
270                     arr_3001.get(j_3001+1));
271
272                 arr_3001.set(j_3001+1,
273                     temp_3001);
274             }
275         }
276
277         txtHasil_3001.append(
278             "Pass " + (i_3001+1) + " : " +
279             tampilNama_3001(arr_3001) + "\n");
280     }
281 }
282
283 public static void main(String[] args) {
284
285     SwingUtilities.invokeLater(() -> {
286
287         new SortingGUI_2511533001().setVisible(true);
288     });
289 }
290 }

```

Penjelasan:

Class SortingGUI_2511533001 merupakan program GUI Java yang digunakan untuk mengimplementasikan proses sorting data mahasiswa menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort. Program menggunakan ArrayList untuk menyimpan data

mahasiswa dan komponen Swing seperti JFrame, JTextField, JButton, JComboBox, dan JTextArea.

Pada constructor, program membuat tampilan GUI yang terdiri dari panel input, area hasil, dan panel kontrol sorting. Selain itu terdapat data awal mahasiswa yaitu Zaky, Digo, Karim, Endy, dan Fahri beserta NIM dan program studinya.

Method tampilData_3001() digunakan untuk menampilkan seluruh data mahasiswa pada area visualisasi. Sedangkan method tampilNama_3001() digunakan untuk menampilkan nama mahasiswa saja selama proses sorting berlangsung.

Method insertionSort_3001() bekerja dengan mengambil satu data kemudian menyisipkannya ke posisi yang sesuai pada bagian data yang sudah terurut.

Method selectionSort_3001() bekerja dengan mencari data terkecil pada setiap iterasi kemudian menukarnya dengan posisi awal.

Method bubbleSort_3001() bekerja dengan membandingkan dua data yang berdekatan lalu menukarnya apabila urutannya salah sampai seluruh data terurut.

Output:

```
=== INSERTION SORT ===
```

```
Data Awal : [Zaky, Digo, Karim, Endy, Fahri]
```

```
Langkah 1 : [Digo, Zaky, Karim, Endy, Fahri]
```

```
Langkah 2 : [Digo, Karim, Zaky, Endy, Fahri]
```

```
Langkah 3 : [Digo, Endy, Karim, Zaky, Fahri]
```

```
Langkah 4 : [Digo, Endy, Fahri, Karim, Zaky]
```

```
=== HASIL AKHIR ===
```

```
Digo (2511523017 - Sistem Informasi)
```

```
Endy (2511531017 - Informatika)
```

```
Fahri (2511511007 - Teknik Komputer)
```

```
Karim (2511512029 - Teknik Komputer)
```

```
Zaky (2511533001 - Informatika)
```

```
=== SELECTION SORT ===
```

```
Data Awal : [Zaky, Digo, Karim, Endy, Fahri]
```

```
Pass 1 : [Digo, Zaky, Karim, Endy, Fahri]
```

```
Pass 2 : [Digo, Endy, Karim, Zaky, Fahri]
```

```
Pass 3 : [Digo, Endy, Fahri, Zaky, Karim]
```

```
Pass 4 : [Digo, Endy, Fahri, Karim, Zaky]
```

```
=== HASIL AKHIR ===
```

```
Digo (2511523017 - Sistem Informasi)
```

```
Endy (2511531017 - Informatika)
```

```
Fahri (2511511007 - Teknik Komputer)
```

```
Karim (2511512029 - Teknik Komputer)
```

```
Zaky (2511533001 - Informatika)
```

=== BUBBLE SORT ===

Data Awal : [Zaky, Digo, Karim, Endy, Fahri]

Pass 1 : [Digo, Karim, Endy, Fahri, Zaky]

Pass 2 : [Digo, Endy, Fahri, Karim, Zaky]

Pass 3 : [Digo, Endy, Fahri, Karim, Zaky]

Pass 4 : [Digo, Endy, Fahri, Karim, Zaky]

=== HASIL AKHIR ===

Digo (2511523017 - Sistem Informasi)

Endy (2511531017 - Informatika)

Fahri (2511511007 - Teknik Komputer)

Karim (2511512029 - Teknik Komputer)

Zaky (2511533001 - Informatika)

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum mengenai implementasi Sorting dan GUI pada Java, dapat disimpulkan bahwa algoritma sorting merupakan metode yang digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diproses. Pada praktikum ini digunakan beberapa metode sorting seperti Insertion Sort, Selection Sort, dan Bubble Sort yang masing-masing memiliki mekanisme berbeda dalam melakukan proses pengurutan data mahasiswa berdasarkan nama.

Selain mempelajari algoritma sorting, praktikum ini juga menerapkan Graphical User Interface (GUI) menggunakan Java Swing sehingga program dapat dijalankan melalui tampilan visual yang lebih interaktif dan mudah digunakan. Komponen GUI seperti JFrame, JButton, JTextField, JComboBox, dan JTextArea digunakan untuk membuat antarmuka program, menerima input data, serta menampilkan proses dan hasil sorting secara langsung kepada pengguna.

Penerapan sorting pada program GUI ini menunjukkan bahwa algoritma pengurutan tidak hanya dipahami secara teori, tetapi juga dapat diimplementasikan dalam aplikasi sederhana yang bersifat interaktif. Visualisasi proses sorting membantu pengguna memahami bagaimana data diproses dan diurutkan langkah demi langkah hingga menghasilkan data yang terurut dengan benar.

Dengan demikian, praktikum implementasi Sorting dan GUI pada Java tidak hanya membantu memahami konsep algoritma pengurutan, tetapi juga memberikan pemahaman mengenai pembuatan aplikasi berbasis GUI, penggunaan ArrayList, event handling, serta penerapan pemrograman berorientasi objek dalam pengembangan program Java sederhana.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn, sehingga mahasiswa memiliki waktu yang cukup untuk mempelajari dan mempersiapkan diri sebelum pelaksanaan praktikum.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Sorting Algorithms in Java". [Online]. Available: <https://www.geeksforgeeks.org/sorting-algorithms/>